ELI — Agent Algorithms

ELI v2.5.11

October 2026

Contents

ELI Agent Algorithms — what each of the 15 agents actually computes **1**
Roles at a glance . . . 1
The 15 specialist algorithms . . . 2
How the DAG sequences the RAG (your "search → summarize → check") . . . 4
Aspirational next steps (genuinely novel combinations) . . . 4
`SpecAgent` and the measurable-agent contract . . . 4

ELI Agent Algorithms — what each of the 15 agents actually computes

> **Updated 2026-09-30.** Orchestrator runs for all CHAT modes; bus composed at S06. `_aggregate_confidence` and `critic`'s corroboration score both gained a query-relevance term.

Your framing is the right lens: **the DAG structures the reasoning steps; RAG is the engine that pulls the raw information during those steps.** ELI maps onto this cleanly —

- **DAG layer (structure):** the `AgentOrchestrator` 12-stage pipeline + the `AgentBus` topological layers (`eli/core/dag.py`) + the coding subtask DAG. This is the "search → summarize → check" skeleton.
- **RAG layer (raw information):** a *subset* of the agents are retrievers (`memory`, `knowledge_graph`, `file_code`) plus the orchestrator's own HyDE→FAISS→RAG→rerank pipeline. These pull evidence.
- **Effectors / reporters:** the rest act (`system`, `plugin`) or report grounded runtime facts (`capability`, `introspection`, `frontier`, …).

So there are **15 specialist agents in `_ALL_AGENTS`** plus the **`AgentOrchestrator`** that sequences them. `SpecAgent` instances and custom agents register on top.

Roles at a glance

Role	Agents
Retriever (RAG engine)	`memory`, `knowledge_graph`, `file_code`
Effector (acts on the world)	`system`, `plugin`
Grounded reporter (read-only facts)	`capability`, `voice`, `introspection`, `frontier`, `reflection`, `proactive`, `self_improvement`, `habit`
Planner (emits structure)	`orchestrator` (bus-level, descriptive) + the real `AgentOrchestrator`

Every agent shares the same skeleton: a **relevance gate** (cheap keyword/action test → skip with `confidence=0` if irrelevant, so the bus stays fast) → its algorithm → an `AgentResult(ok, confidence, data)`. The

bus runs them in **topological layers** with **per-agent hard timeouts**, then `_aggregate_confidence` fuses them: `contribution = evidence_quality × evidence_density × learned calibration × query relevance`, single-agent-capped, corroboration-bonused. The query-relevance term (2026-09-29, distinct from the per-agent gate above) is `relevance_gate(term_overlap(user_input, representative_text))` — a fluent, evidence-dense result with nothing to do with what was actually asked no longer scores as highly as an on-topic one; `CriticAgent`'s corroboration score gets the same discount (two sources agreeing with each other doesn't mean either is on-topic). Env `ELI_AGENT_BUS_RELEVANCE_GATE` disables it.

The 15 specialist algorithms

1. `memory` — hybrid dense+sparse retriever (RAG core) · 5.0s

- **Gate:** `_eli_memory_should_run` (explicit memory actions, memory markers, or ≥4-word CHAT; blocks stale context bleeding into commands).
- **Algorithm:** `recall_memory` = **FAISS vector search first** (dense), with **FTS5/LIKE keyword search** as a supplement on cold/short queries (sparse); noise-filtered (drops ELI's own reflections/assistant-insights), importance- ordered. Plus `search_conversations` (FTS5), recent turns, session summaries.
- **Out:** `{results, conv_hits, memory_context}`. The primary RAG retriever.

2. `system` — deterministic action effector · 8.0s

- **Gate:** `action ∈ SYSTEM_ACTIONS` (121 OS and runtime actions); `LLM_ACTIONS` (GENERATE_SCRIPT, etc.) are **deferred** (never run inside the timed parallel phase — they'd time out + double-execute).
- **Algorithm:** delegates to `executor_enhanced.execute(action, args)` behind the security allowlist/sandbox. Confidence 0.92 ok / 0.20 fail. The hands.

3. `habit` — pattern-store reader · 3.0s

- **Gate:** habit/routine/schedule keywords or `OPEN_APP`.
- **Algorithm:** reads `habit_rules` (all) + `habit_events('app_launch', 14d)`; confidence by rule presence. (Frequency mining itself lives in the proactive daemon; the agent surfaces the rules + event volume.)

4. `self_improvement` — failure-cluster surfacer · 3.0s

- **Gate:** self/improve/fix/learn keywords or SELF_* actions.
- **Algorithm:** `analyze_failures(7d, clustered by occurrence_count)` + pending capability proposals. Feeds the self-upgrade loop (see `background_tasks.md`).

5. `proactive` — artifact reader · 3.0s

- **Gate:** proactive/morning-report/insight keywords or PROACTIVE_* actions.
- **Algorithm:** reads `artifacts/proactive/latest_{context,summary,action}.txt` – `PROACTIVE_STATUS`. Surfaces the background daemon's findings.

6. `frontier` — cross-system introspection report · 5.0s

- **Gate:** FRONTIER_STATUS / ELI_IDENTITY_AUDIT or "full system status".
- **Algorithm:** `build_frontier_status_report` / `eli_identity_audit` — an import + wiring matrix across runtime/memory/self/proactive/image/world/labs. Deterministic whole-system audit.

7. `plugin` — plugin effector · 6.0s

- **Gate:** `action ∈ PLUGIN_ACTIONS` (ADD_EVENT, GET_WEATHER, LIST_EVENTS, REMOVE_EVENT).
- **Algorithm:** `execute(action, args)` into the plugin layer; silent skip otherwise.

8. `capability` — capability-manifest reader · 6.0s

- **Gate:** capability/awareness/"what can you do" or LIST_CAPABILITIES etc.
- **Algorithm:** `execute(LIST_CAPABILITIES | AWARENESS_STATUS | CODE_CHANGES)` → grounded capability surface (no confabulated feature lists).

9. `voice` — config reporter · 5.0s

- **Gate:** voice/tts/stt/mic keywords.
- **Algorithm:** reads `ELI_TTS_ENGINE/ELI_PIPER_MODEL/ELI_MUTE` + probes `faster_whisper` import. Reports, never emits audio (that's the TTS router).

10. `orchestrator` (bus-level) — descriptive planner · 3.0s

- **Gate:** multi-step trigger groups (e.g. {document, save, open}) or GROUNDED_SYNTHESIS_ACTIONS.
- **Algorithm:** emits a structured `plan` dict (now also surfaced as the typed `ExecutionPlan`). **Excluded from confidence aggregation** (it plans, it isn't evidence). The *executing* planner is the ReAct loop in `AgentOrchestrator`.

11. `file_code` — source-code RAG (grep retriever) · 4.0s

- **Gate:** grounded query or code/file/pipeline/db keywords.
- **Algorithm:** builds query-derived regex patterns, **greps a canonical `files_map`** of ELI's own modules (agent_bus, engine, orchestrator, memory, router, …), returns `file:line: content` snippets. Lets ELI answer "which file does X" from real source, not memory.

12. `reflection` — reflection-log reader · 4.0s

- **Gate:** reflection/pattern/noticed/insight keywords or grounded query.
- **Algorithm:** recent `observations` (≤8) + session summaries (≤3) → insight list. ELI's "what I've noticed about how you use me."

13. `introspection` — self-model reporter · 4.0s

- **Gate:** architecture/pipeline/"how do you work" or EXPLAIN_*_RUNTIME.
- **Algorithm:** `IntrospectionAgent → get_pipeline() + get_memory_stats() + get_runtime()` → grounded pipeline description. Powers honest self-explanation.

14. `knowledge_graph` — graph retriever (structured RAG) · 3.0s

- **Gate:** non-empty KG.
- **Algorithm:** `context_for_prompt(query)` over `kg_entities/kg_relations` (FTS5 fuzzy entity match + relation context). **DAG edge `memory → knowledge_graph`:** it now seeds the query with `intent["_upstream"]["memory"]` hits (topological layering), so the graph lookup is conditioned on what memory just surfaced.

15. `critic` — second-tier verifier (corroboration check) · 2.0s

- **Gate:** ≥2 grounded upstream sources — self-gates to nothing on trivial turns, so it never fires when there's nothing to cross-check.

- **Algorithm:** the one bus agent that reasons over **other agents' output** (`intent["_upstream"]`) rather than the raw query — **deterministic, no LLM**. It pulls the representative text each retriever surfaced this turn, measures **pairwise term agreement**, and emits a **corroboration-vs-contradiction** signal. Corroboration alone isn't enough (2026-09-29): two sources can agree strongly with EACH OTHER while both being off-topic for what was actually asked, so on the non-contradiction branch, agreement is additionally discounted by how well the sources match the *query* (`relevance_gate` over their average `term_overlap(user_input, text)`), not just each other. `_representative_text` moved from a `CriticAgent` staticmethod to a module-level function, since `_aggregate_confidence` now needs it too — one owner. **DAG edges:** depends on `memory`, `file_code`, `knowledge_graph` and `system`, so it runs in a **downstream topological layer** and sees their results — giving the agent DAG a real second tier instead of a flat fan-out.

How the DAG sequences the RAG (your "search → summarize → check")

1. **Route** → action + reasoning mode.
2. **`AgentOrchestrator`** runs for **all CHAT modes** (gradient depth) and for non-CHAT actions; for non-CHAT it dispatches the **bus** then runs a **ReAct loop** (tool → observe → decide → next tool) — the executing DAG of steps.
3. **`AgentBus`** builds the **agent DAG** (`_AGENT_DEPENDENCIES`), runs agents in **topological layers**, passing each layer's results downstream (`memory` → `knowledge_graph`). Retrievers pull evidence; reporters ground it; effectors act.
4. **Grounding/evidence layer** gates the synthesis (`output_violates_evidence`).
5. **Coding tasks** decompose into a **subtask DAG** (`plan_graph`) and heavy ones run on **background workers** — see `dag.md` + `background_tasks.md`.

That is the literal realisation of "DAG structures the autonomous reasoning steps while RAG pulls the raw information."

Aspirational next steps (genuinely novel combinations)

These are the "never-before-put-together" moves the current architecture is one step away from:

1. **Bias/verification agent as an explicit DAG layer.** [done] **Shipped** — this is now the `critic` agent (#15 above): it depends on the retrievers (`memory`/`system`/ `knowledge_graph`) and runs *after* them as a topological layer — literally "search → summarize → **check for corroboration/contradiction**" — feeding the grounding signal. (Originally proposed here as future work; now built.)
2. **A `web` retriever agent** (WEB_SEARCH as a first-class RAG node) so external retrieval joins memory/KG/file_code in the same layer, with the verify layer cross-checking sources.
3. **Calibration-aware selection.** The bus already learns per-(agent,action) calibration; feed it back into *selection* so chronically-zero agents are dropped from the DAG for that action (closes the learned-trust/unlearned- selection gap noted in `orchestration_and_agents.md`).
4. **Bus↔coding bug-memory unification.** The coding engine's `bug_memory` and the agents' `self_improvement` failure log are two halves of one "what went wrong and how we fixed it" store — merging them gives ELI a single experiential memory across chat and code.
5. **Orchestrator stage-dict → typed `ExecutionPlan` DAG.** Fold the engine's rich stage plan into `ExecutionPlan.steps` as a true DAG so one plan type drives the whole pipeline end-to-end.

`SpecAgent` and the measurable-agent contract

A further agent class joins the fifteen built-ins. `SpecAgent` is not hand-written Python — it is an interpreter for an `AgentSpec` (see `orchestration_and_agents.md`), and its algorithm is deliberately small:

```
run(user_input, intent):
```

```
    if not spec.should_run(user_input, intent.action):     # trigger gate
        return skipped                                     # costs one regex/substring
    for cap in spec.permissions:                           # capability gate
        if not permissions.check(agent, cap).allowed:
            return denied
    output = broker.infer(prompt=user_input,
                          system=spec.system_prompt,
                          max_tokens=spec.max_tokens,
                          temperature=spec.temperature,
                          background=True)
    verdict = spec.evaluate(output)                        # the measure
    return AgentResult(ok=verdict.ok,
                       confidence=verdict.score,
                       data={"content": output if verdict.ok else ""})
```

Three properties worth stating:

- **The trigger gate runs first and is cheap.** An untriggered turn costs a substring or regex test, not an inference call.
- **Confidence is earned, not asserted.** Every other agent picks its own confidence number. A `SpecAgent`'s confidence *is* its score against its own declared success criteria — the fraction of checks its output actually passed.
- **Failing its own test yields no evidence.** `data["content"]` is emptied when the verdict fails, so a misbehaving custom agent cannot feed unchecked text into the bus's evidence fusion. This directly addresses the long-standing weakness that evidence density "only counts known keys", by making the agent's own contract the arbiter.

Because a spec contains no code, `SpecAgent` bypasses the custom-agent trust chain entirely — there is nothing to hash, scan or approve.